SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	A. Mohith Krishna	**Reg. No.:**	192425199
Section / Batch:	2024	**Date:**	30/08/2026

Code of Conduct

I A. Mohith Krishna (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Mohith

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A conversational AI system must identify grammatical relationships in user queries containing long-distance dependencies. The developers are comparing constituency parsing and dependency parsing.

Analyze how the two representations differ in capturing hierarchical and word-to-word relationships, and justify which approach is more suitable for extracting meaningful relationships from conversational text.

2. An NLP system processes partially typed search queries such as “best hotels near…” and “book a flight from Chennai to…”. The system currently uses conventional top-down parsing, while the developers are considering Earley parsing.

Analyze how the two approaches behave when input is incomplete or ambiguous, and justify which parsing strategy is more appropriate for interactive NLP applications.

3. A grammar-checking system must distinguish between multiple interpretations of structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

Analyze how each approach represents and resolves syntactic ambiguity, and justify which approach would provide the best balance between interpretability and practical accuracy.

4. A multilingual NLP system must process sentences in which grammatical information depends on number, gender, tense, and person. The developers are considering feature structures and traditional context-free grammar rules.

Analyze how feature structures extend basic grammar representations and justify their usefulness for enforcing grammatical constraints in multilingual systems.

5. A voice assistant must interpret commands containing verbs with different argument requirements, such as “give the book to John,” “sleep,” and “put the file on the desk.” The developers are considering subcategorization frames.

Analyze how subcategorization information helps determine valid sentence structures and justify its importance in semantic and syntactic analysis.

1. Constituency Parsing vs Dependency Parsing

Analysis

Constituency parsing represents a sentence as a hierarchical structure of phrases. It groups words into constituents such as **Noun Phrase (NP), Verb Phrase (VP), and Prepositional Phrase (PP)**.

For example:

Sentence:
The user who called yesterday wants help.

A constituency parser identifies structures such as:

```
S
├── NP
│   ├── The
│   └── user
└── VP
    ├── ...
    └── wants help
```

It is useful for understanding the hierarchical organization of a sentence.

Dependency parsing, on the other hand, represents relationships directly between individual words. Each word is connected to its syntactic head.

For example:

user ──subject──> wants
help ──object───> wants
called ──modifier──> user

Therefore:

Constituency Parsing	Dependency Parsing
Represents phrase hierarchy	Represents word-to-word relationships
Uses constituents such as NP and VP	Uses relations such as subject and object
Good for phrase structure	Good for relationship extraction
Can be complex for long-distance relations	Directly connects related words

Python Code

```
import spacy

nlp = spacy.load("en_core_web_sm")

sentence = "The user who called yesterday wants help."
doc = nlp(sentence)

print("Word -> Head : Relationship")

for token in doc:
    print(f"{token.text} -> {token.head.text} : {token.dep_}")
```

Expected Output

```
The -> user : det
user -> wants : nsubj
who -> called : nsubj
called -> user : relcl
yesterday -> called : npadvmod
wants -> wants : ROOT
help -> wants : dobj
```

Justification

For conversational AI, dependency parsing is more suitable for extracting meaningful relationships because it directly identifies relationships such as subject, object, modifier, and complement.

This is particularly useful when conversational sentences contain long-distance dependencies, because related words can be connected directly even when other words occur between them.

Conclusion

Constituency parsing is better for understanding hierarchical phrase structures, while dependency parsing is better for identifying direct grammatical relationships. Therefore, dependency parsing is preferred for relationship extraction in conversational text.

2. Top-Down Parsing vs Earley Parsing

Analysis

Top-down parsing starts from the start symbol of a grammar and tries to generate the input sentence by applying grammar rules.

For example:

S → NP VP
NP → Det N
VP → V NP

The parser begins with S and works downward toward the input words.

However, when the input is incomplete, such as:

> **"best hotels near..."**

the parser may not be able to complete the expected structure.

Similarly:

> **"book a flight from Chennai to..."**

is incomplete because the destination has not yet been entered.

Earley parsing is a dynamic-programming parsing method that maintains partial parsing states. It uses three main operations:

1. **Prediction**
2. **Scanning**
3. **Completion**

It can handle **incomplete and ambiguous structures** more flexibly.

Top-Down Parsing	Earley Parsing
Starts from grammar's start symbol	Maintains partial parsing states
Works well with predictable input	Works well with incomplete input

May require backtracking	Handles ambiguity efficiently
Less suitable for interactive input	Suitable for interactive NLP

Python Code

```
from lark import Lark

grammar = """
start: "book" "a" "flight" "from" CITY "to" CITY
CITY: "Chennai" | "Delhi" | "Mumbai"
"""

parser = Lark(grammar, parser="earley")

sentence = "book a flight from Chennai to Delhi"

tree = parser.parse(sentence)

print(tree.pretty())
```

Expected Output

```
start
  Chennai
  Delhi
```

Justification

Interactive applications receive input **incrementally**, meaning the user may stop typing at any point.

For example:

book a flight from Chennai to...

Earley parsing can preserve the partial grammatical structure and continue parsing when the user provides the next word.

It can also maintain multiple possible interpretations when the input is ambiguous.

Conclusion

Top-down parsing is suitable for simple and complete grammatical input. However, Earley parsing is more appropriate for interactive NLP applications because it can handle incomplete, ambiguous, and incrementally entered input.

3. CFG vs PCFG vs Neural Constituency Parser

Analysis

A **Context-Free Grammar (CFG)** uses predefined grammar rules to generate syntactic structures.

Example:

S → NP VP

NP → Det N

VP → V NP

If a sentence is structurally ambiguous, CFG may produce **multiple valid parse trees**, but it does not determine which interpretation is more likely.

A **Probabilistic Context-Free Grammar (PCFG)** extends CFG by assigning probabilities to grammar rules.

For example:

VP → V NP [0.7]

VP → VP PP [0.3]

When multiple parses are possible, the parser can use probabilities to select the **most likely parse**.

A **neural constituency parser** uses machine-learning models, often trained on large annotated datasets, to learn syntactic patterns from context. It can resolve complex ambiguities more accurately but is generally less interpretable.

Comparison

Feature	CFG	PCFG	Neural Parser
Grammar rules	Manual	Manual + probabilities	Learned

Handles ambiguity	Produces alternatives	Ranks alternatives	Predicts likely structure
Interpretability	High	High	Lower
Context understanding	Limited	Moderate	High

Python Code – CFG Ambiguity

```
from nltk import CFG

from nltk.parse import ChartParser

grammar = CFG.fromstring("""

S -> NP VP

NP -> Det N | NP PP

VP -> V NP | VP PP

PP -> P NP

Det -> "the"

N -> "man" | "telescope"

V -> "saw"

P -> "with"

""")

parser = ChartParser(grammar)

sentence = "the man saw the telescope with the telescope"

for tree in parser.parse(sentence):

    print(tree)
```

Expected Result

The CFG can generate **more than one possible parse tree**, showing that the sentence is structurally ambiguous.

Justification

- **CFG** is highly interpretable but cannot effectively rank ambiguous interpretations.
- **PCFG** adds probabilities and can select the most probable interpretation while keeping grammar rules understandable.
- **Neural parsers** generally provide better practical accuracy but are less transparent.

For a grammar-checking system where both accuracy and explanation are important, **PCFG provides a good balance**.

4. Feature Structures vs Traditional CFG

Analysis

Traditional **Context-Free Grammar (CFG)** describes syntactic structures using rules such as:

S → NP VP

However, basic CFG does not directly represent grammatical properties such as:

- Number
- Gender
- Person
- Tense
- Case

Feature structures extend grammar rules by attaching attributes and values to linguistic units.

For example:

NP:

Number = Singular

Person = 3

Gender = Male

A verb can also have features:

VP:

Number = Singular

Person = 3

Tense = Present

The system can then check whether the features agree.

Python Code

```
def check_agreement(subject, verb):
    if (subject["number"] == verb["number"] and
        subject["person"] == verb["person"]):
        return "Grammatically valid"
    return "Agreement error"
subject = {
    "number": "singular",
    "person": 3,
    "gender": "male"
}
verb = {
    "number": "singular",
    "person": 3,
    "tense": "present"
}
print(check_agreement(subject, verb))
```

Output

```
Grammatically valid
```

If the verb is changed to plural:

verb["number"] = "plural"

the output becomes:

Agreement error

Justification

Feature structures are useful because they allow the system to enforce grammatical constraints.

For example:

Subject:

Number = Singular

Person = 3

Verb:

Number = Singular

Person = 3

The features match, so the sentence is valid.

This is especially useful in **multilingual NLP**, where grammatical agreement may depend on number, gender, person, tense, and other linguistic properties.

Conclusion

Feature structures extend traditional CFG by adding grammatical attributes and constraints. They reduce the need for many separate grammar rules and are therefore highly useful for multilingual grammatical analysis.

5. Subcategorization Frames

Analysis

Subcategorization frames describe the arguments that a verb requires to form a valid sentence.

Different verbs have different argument requirements.

Examples

1. Sleep

sleep → Subject

Example:

The child sleeps.

2. Give

give → Subject + Object + Recipient

Example:

She gives the book to John.

3. Put

put → Subject + Object + Location

Example:

Put the file on the desk.

Therefore, a voice assistant can use subcategorization information to determine whether the required arguments are present.

Python Code

```
subcategorization = {
```

```
    "sleep": ["subject"],
    "give": ["subject", "object", "recipient"],
    "put": ["subject", "object", "location"]
}
verb = "give"
arguments = ["subject", "object", "recipient"]
required = subcategorization[verb]
if all(arg in arguments for arg in required):
    print("Valid sentence structure")
else:
    print("Invalid sentence structure")
```

Output

Valid sentence structure

For example, if:

```
verb = "put"
arguments = ["subject", "object"]
```

the output will be:

Invalid sentence structure

because put requires a location.

Application to Voice Assistant

For:

> "Give the book to John."

The system identifies:

Verb → give
Object → book
Recipient → John

For:

> "Put the file on the desk."

The system identifies:

Verb → put
Object → file
Location → desk

This information helps the voice assistant understand what action needs to be performed and what entities are involved.

Justification

Subcategorization frames are important because they connect syntactic structure with semantic meaning. They help an NLP system:

- Identify required arguments.
- Detect incomplete sentences.
- Detect invalid verb usage.
- Identify semantic roles.
- Convert commands into executable actions.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1	3	3	3	3	4		78

2	3	3	4	3	3		
3	3	3	3	4	3		
4	4	3	3	3	3		
5	3	4	3	3	3		

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____